

•



• Concepts Glossary

On this page

Concepts Glossary

Was this page helpful? 

Acquirer

An acquirer represents the bank or the financial institution that acquires money from the client on behalf of the merchant as a payment for products or services.

In an every day scenario the acquirer is responsible for handling the merchant's transactions and collect payments from the clients [issuers](#).

Action

Actions are asynchronous flags that are raised on events when some predefined condition is met.

For example the **Blocked** action is raised when a fraudulent transaction is found.

Administrator🔗🔗

Pulse user that has admin role.

This user can:

- **Monitor and Control** Pulse services.
- **Manage** security configuration.
- **Manage** Pulse's configuration attributes,
- **View** Pulse's log and general information.

Aggregation🔗🔗

An aggregation is the calculation of a [metric](#) for a combination of dimensions 🔗 i.e. one or more fields that are used to [group](#) the data into sets, each of which gets its own aggregation value.

For example, if the dimensions are State and Bank, the aggregation will be calculated for all the possible combinations of the available states and banks. If there are 50 states and 10 banks in the dataset, the aggregation needs to be calculated 500 times.

AppEngine🔗🔗

An AppEngine is a conceptual group of services that run on their own process.

Services that run on an AppEngine are said to run on an **AppEngine Context**.

Batch🔗🔗

The amount of [events](#) to be processed at the same time.

By default the events are processed one by one. However, most use-cases don't need a sub-second real-time resolution (where resolution is the max timeframe in which to report notifications). It's very common that a resolution of one second, or even more, is sufficient. Given that the current latency to process an event is low (sub-millisecond), multiple events can be processed at the same time, in **batches**. Batch option for stream definition, for example, serves for this purpose.

Chargeback🔗🔗

A chargeback happens when a transaction made previously between an customer and a merchant is returned to the customer.

While this can happen in many different use cases, such a case would be making a return after purchasing an item in a store. Here the merchant will make a chargeback of the transaction made previously. It's basically a reversal mechanism for transactions.

Clock🔗🔗

An important Pulse property that defines how a [timestamp](#) is generated for each event injected to [stream](#).

If you are already using Feedzai Pulse interface you might already be familiar with choosing option for clock.

In this context three types of clock mean the following:

1. **Event-based**-means that each event itself contains the timestamp
2. **System clock**- means that when the event arrives, the server marks it with the current time of the machine
3. **Stream clock**- means that when an event arrives its time is set according to the clock of the corresponding stream

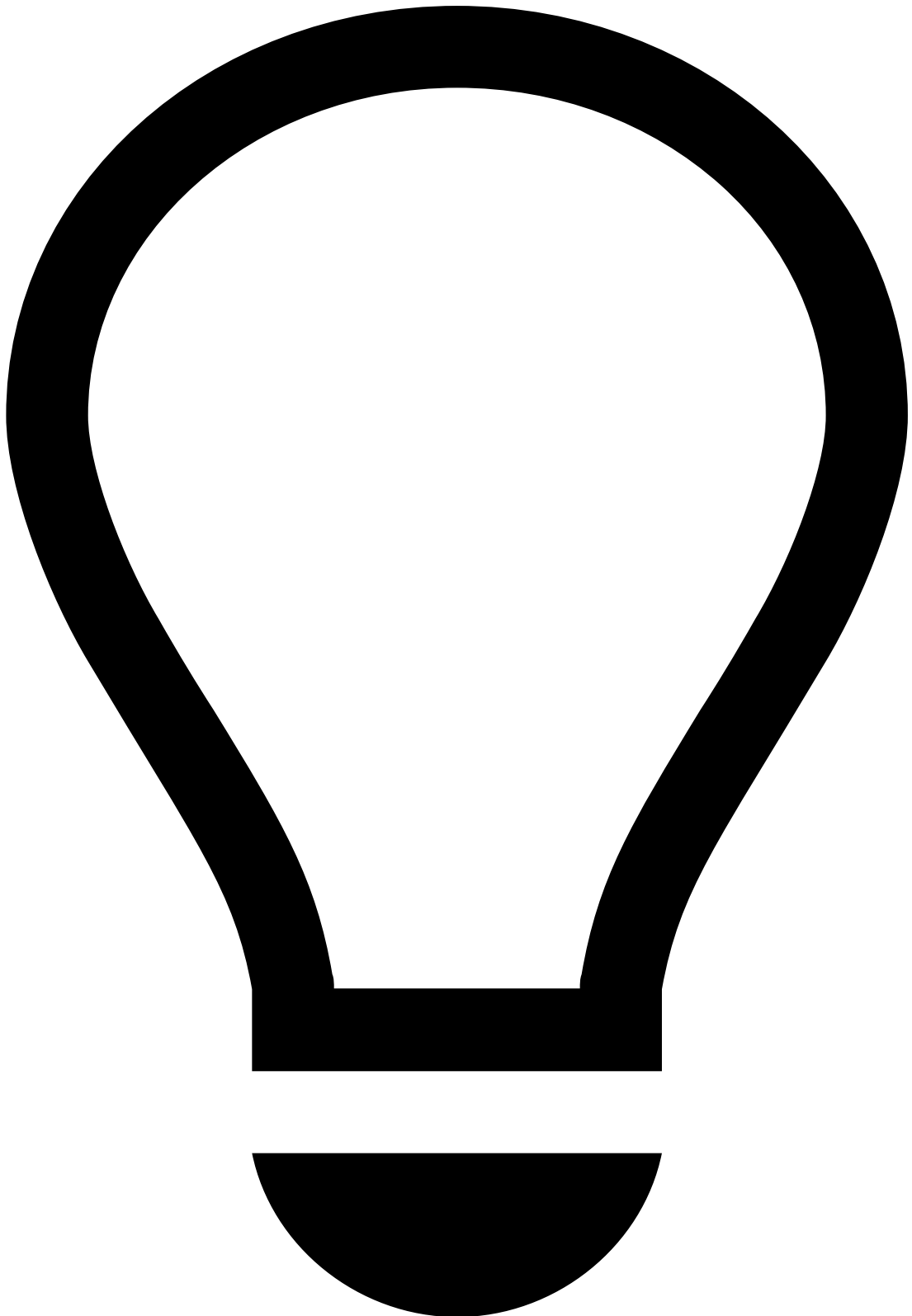
Cluster

All the Pulse software components that are connected to Zookeeper, these include Pulse Servers, Application Engines and external Batch Jobs. Each cluster lives in a single Data Center, and when referring to a Pulse deployment spanned across multiple Data Centers the used term is usually Global Cluster, but it will nonetheless be made clear.

CNP

Card Not Present transaction represents a payment card transaction that was made without the customer being physically present with the card.

Payments made over Internet are one example of CNP transactions since both the customer and the card wasn't physically present in the POS when the transaction was made. These types of transactions have a much higher rate of credit card fraud than their CP counterpart, mainly for the easiness of using counterfeit or stolen credit card data.



CP

Inversely, [Card Present](#) transactions represent payment card transactions where the card was physically available when the purchase was made.

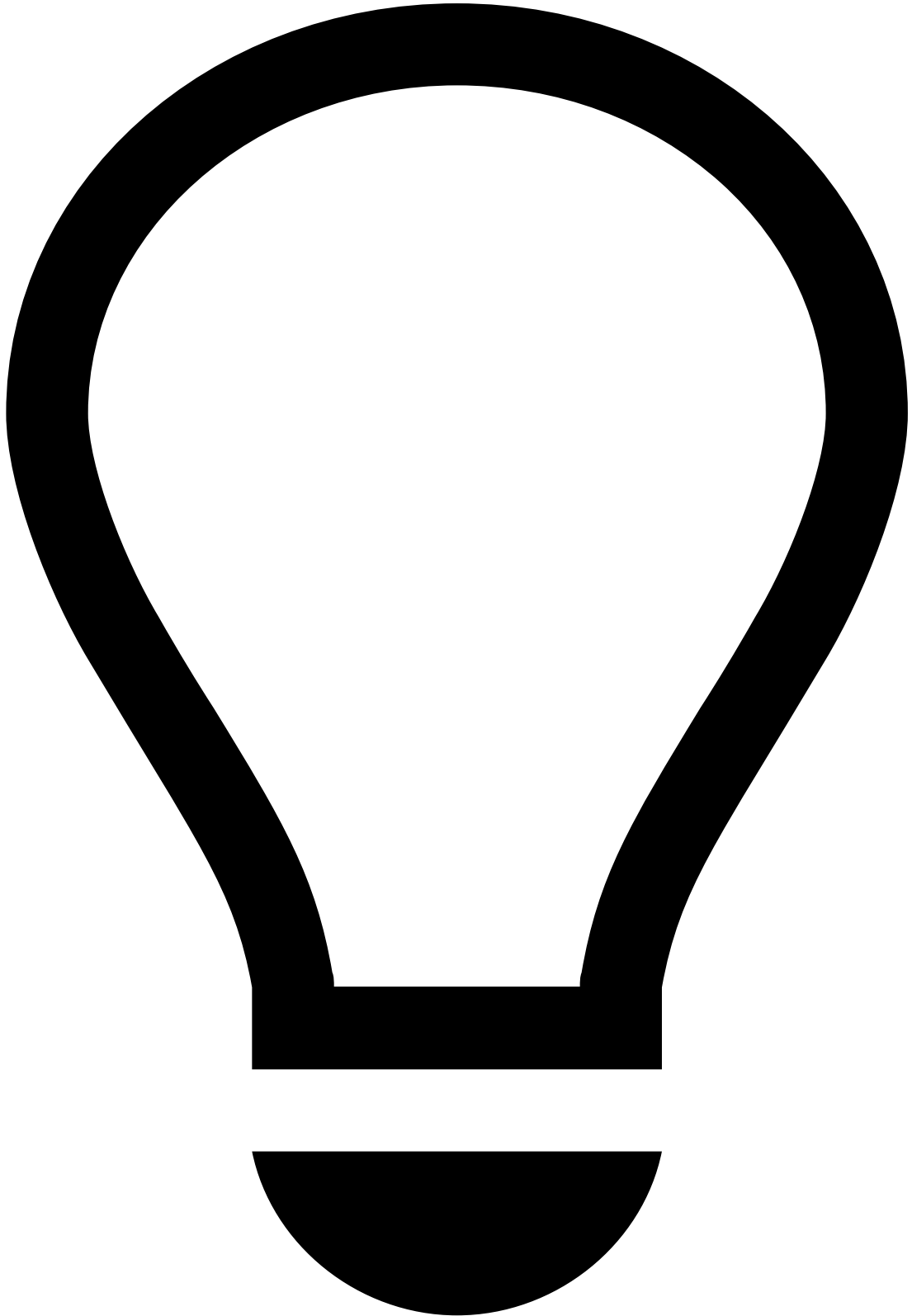
Control Flow

Control flow is a mechanism by which it is possible for a computer program to make choices.

In the context of a [Workflow](#), Control Flow are the mechanisms that allow to control how an event progresses through the Workflow Elements, namely if it should skip none, some or all of the Workflow Elements that follow.



Card Present transactions represent payment card transactions where the card was physically available when the purchase was made.



Inversely, [Card Not Present](#) transaction represents a payment card transaction that was made without the customer being physically present with the card.

Data Center🔗🔗

A logical and physical relation of nodes. Each Data Center has its own cluster and must have all the components of the system to work by itself. This means that it follows a shared-nothing approach in order to support scenarios of disaster recovery where a single Data Center must be able to fully operate without need of a system that belongs to another.

Dead Letter🔗🔗

Built-in [Action](#) that is raised when an event reaches the end of its path in the [Workflow](#).

This [Action](#) is used by the Event Storage Service to store the event in persistent storage.

Dictionary🔗🔗

Is a data-type which is similar to a map or a hash table in a typical programming language. It also maps keys to values. In PulseQL the keys would be the groups and the values are the aggregator results.

So, for example, for the transactions stream example, if we create a dictionary grouped by bank, the keys are names of banks and the values can be the average amounts. A snapshot of this dictionary could look like this;

bank	avg_amount
Bank A	\$ 50
Bank B	\$ 30
Bank C	\$ 65.5

Dictionaries support lots of operations. You can, for example, filter them, project their entries, aggregate them or lookup the value of a single entry.

Dimension🔗🔗

A dimension is an analysis axis that contains a limited set of values from which some can be selected, thus drilling in into that sub set. Along with Dimension Mappings they can transform machine codes into human readable information: "0" s and "1" s into "United States of America" and "Portugal" .

For example the total USA transactions could be at a given time 150K, drilling in in this case could be choosing to view the transactions only for the state of New York (which could be 10K). We could drill even further, if we had, for example, a bank dimension. In this case we would only see the transactions for a given bank in New York (which could be 2K).

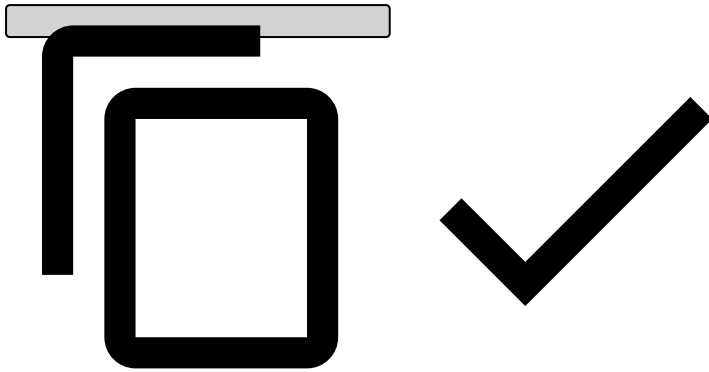
Event🔗🔗

A single *packet* of data within a [stream](#); also called a "tuple". An event consists of [fields](#) with associated values, conforming to the stream's data [schema](#).

The following is an example of a tuple for a stream of [transactions](#):

```
{
  timestamp:    1,
  state:        'Alaska',
  bank:         'Bank of America',
  rejected:     1000,
  duration:     15.5,
  amount:       1000,
  operation_type: 'Withdrawals',
  card_type:    'Visa',
}
```

```
client_type: 'ATM'
}
```



Event Processing¹

A set of technologies designed to process high-volumes of continuously arriving data in real-time.

In this context, continuously means that data may arrive at any moment over the course of arbitrarily long periods of time. For example, an application designed to monitor traffic in a highway system may be required to execute without interruptions for months or even years. During all this time, the application is continuously receiving notifications such as "car entered the highway", analyzing them and producing meaningful results like "there are currently 1000 vehicles inside the highway" or "the vehicle with license plate 11-22-AA drove through the highway at an average speed of 150 km/h". This information could then be displayed in a dashboard where a human operator could look at it and use it to make informed decisions. The important point to retain is that, in this kind of applications, one can't wait for all data to arrive before it is processed. Moreover, in most scenarios, it is desirable to process incoming data as soon as possible so that the resulting analysis is still up-to-date and relevant when it is acted upon.

Explanation¹

An Explanation details why a given transaction reached a certain outcome and got a given [score](#). It contains a list of reasons that help explain the outcome and gives the user further hindsight about the final decision.

Please refer to [this section](#) to learn more about Explanations.

Feature¹

A feature is an attribute of an [instance](#). It represents information necessary for a Machine Learning algorithm to identify a pattern.

Please refer to [this section](#) in order to obtain more details.

Feedzai IQ¹

Feedzai IQ is a suite of ready-to-adopt risk assessment solutions providing AI-powered insights based on aggregated intelligence from across the Feedzai global network.

For more information about Feedzai IQ, please refer to [this section](#).

Field¹

A Field is a component of a data source's [schema](#). In CSV file or other tabular data formats, a field corresponds to a column.

To learn more about fields, refer to the [Data fields](#) section in the documentation about Data Sources.

Group

A virtual subset of members of the cluster that have similar functionality, i.e. all the Pulse Servers belong to a group of pulse servers but the Application Engines belong to another group.

Note that this is a distinct concept from the [Group By](#) operator used in [plans](#).

Instance

Instance is an event that is enriched with the [Features](#) from the [Fraud Model Definition](#).

The instance does not travel the [Workflow](#), it is computed using the event data and then sent to FOS either for training or scoring.

Instance may also be used to refer to an incarnation of a Service like the [Fraud Rules Service](#).

Issuer

An issuer represents the bank or the financial institution that legally emitted the payment card to the cardholder.

In an every day financial transaction between a client and a merchant, the issuer is responsible for issuing the money to the [acquirer](#) bank. He does this on behalf of the client, usually assuming the liability for the client's capacity to pay off the debts.

MCC

Merchant Category Code is a four-digits number assigned to a merchant that represents the type of goods or services it performs.

This code is used to categorize the merchants by business area and activity in order to improve the transaction context. It's easy to understand why, the average transaction amount between a Bus Line merchant (MCC 4131) and a Car Dealership (MCC 5511) will be vastly different.

There are some merchants that have special MCC numbers assigned, the most known case being Airlines.

Member

A software component that is part of a cluster. Core members include Pulse Server and Application Engines.

Merchant

A merchant represents a business person who trades in commodities produced by others with the public, also known as clients. It is usually represented by an [acquirer](#) in a financial transaction.

Metric

A metric (also known as **profile** or **historical profile**) adds new fields to a data source, based on an aggregation of events over a period of time, to track entity behavior patterns.

At its core, a metric performs calculations, called [aggregations](#) —such as sums, counts, maximums, averages, etc.— on a specific data field (e.g. transaction amount), grouped by one or more fields (e.g. a credit card), and over a defined [time window](#) (e.g. the last 24 hours). This allows quantifying and monitoring behavioral patterns, making it easier to detect changes that may signal suspicious activity.

For more information about metrics, refer to the [Metrics](#) section in the documentation about Plans in Workflows.

Node^â

An instance of an operating system. It doesn't relate to physical machines *per se* since several operating system instances can reside in the same physical machine for example by the means of virtualization.

Operation^â

Term used in the context of [Resource Versioning](#) to represent a modification made to a Resource.

POS^â

Point of Sale is the equipment used by a merchant to handle a customer payment inside a store.

Can also be called checkout and it calculates the amount owed by the customer in exchange for goods or services acquired. Typically issues a receipt after the payment was made.

Plan^â

A plan is a recipe of how to augment a data source via data transformations, essentially transforming one data source into another. Plans are typically used to prepare the data to be used to train machine learning models, or to transform the data in the runtime environment for processing via rules or models.

To learn more about plans, refer to the [Plans](#) page.

Profile Update Job (PUJ)^â

A batch process that reads transactions from the database, computes the [metrics](#) using the event processing engine and stores them to a data store.

Reason^â

A Reason represents a fact that was considered by Pulse when scoring a transaction that went through an workflow. It can detail both fraudulent and non-fraudulent behavior and is usually part of an [Explanation](#).

Replication Factor^â

Replication factor is the number of nodes which are running the same component in distinct physical machines. For example, a replication factor of two means that the Pulse Server is replicated across two physical machines as well as the Application Engines.

Resource^â

Represents any entity inside Pulse. Examples of resources include KPIs, Baselines, Lists, Rules, etc.

Schema^â

A data schema is a set of metadata that describes the attributes available in an [event](#), and in particular their [data types](#).

Refer to [Data schemas](#) for a detailed explanation.

Score^â

Score is the value that represents the probability of an event being fraudulent. The higher the score the more likely is that the event is fraudulent.

The score is an integer value between 0 and 1000.

The process of calculating the score is named **scoring**. Please refer to [this section](#) to learn about models and training mechanisms.

Service🔗🔗

Service is an isolated piece of software that provides some kind of functionality as a service to other services.

Settlement🔗🔗

Usually the third and final step in a transaction, the settlement occurs between the issuers and the acquirer about two days after the original transactions were made.

It is in this step that the money is really transferred from the client (issuer) into the merchant (acquirer).

The reasoning behind this is that if a business receives 100 daily payments from bank A, it does not make sense to make each individual transaction right in that moment. The transaction fees would be too high. What happens instead is that these 100 payments will be summed in a single transaction that occurs X time later.

Slack🔗🔗

The amount of time the events can "wait" for each other when they are coming from different sources.

Sometimes you can be interested in receiving events from multiple sources where clocks are almost (but not quite) synchronized. In this case you can define a time slack where the events are re-ordered and processed in-order.

Snapshot🔗🔗

A picture of the current state of data contained in the object requested at a pre-defined interval in time. It contains all the data of that object at that exact moment. It applies to all objects except streams.

Stream🔗🔗

A stream represents a source of real-time data. It's like a connection port between the world and Pulse.

Similar to a table of a database, it is the data structure inside Pulse that defines the schema of the data that will be injected into Pulse.

Tenant🔗🔗

Tenants are the individual entities to which Multi-Tenancy is applied to. They represent separate entities to whom data and certain resources (Rules, Environments and Templates) should be isolated. In order to allow this separation they are associated to users which inherit their isolation.

Tenant Domain🔗🔗

Tenant domains are used to group multiple tenants based on a common domain, they work much like a normal group. It is through them that we specify which Tenants can have resources on a given [workflow](#) element.

Timestamp🔗🔗

The time the event occurs.

Transaction🔗🔗

A transaction is an exchange of goods between two parties.

TrustScore^â

TrustScore is an aggregated risk score that combines the outputs from multiple models trained on relevant patterns from Feedzai's global client base. It uses federated learning techniques to allow financial intelligence sharing without relying on consortium-based data sharing or aggregation, thus ensuring data segregation and confidentiality.

For more information about TrustScore, please refer to [this section](#).

Tuple^â

A synonym for [event](#).

Window^â

Time interval which defines which real-time events from the data stream should be used for calculating aggregations in a [metric](#). For details on the syntax used to define windows, please refer to the [Time window](#) section of the documentation on Plans in Workflows.

There are two major types of windows:

Sliding Window^â

A [sliding window](#) contains the events received within a fixed-size interval. This interval could be temporal or tuple-based. For example, a sliding window with interval equal to 5 minutes will contain only the events received during the past 5 minutes. As soon as the event becomes older than this threshold, it is expired from the window. The window in the example above is a time-based sliding window.

Tumbling Windows^â

Unlike sliding windows, tumbling windows expire all events at once, once they reach their predetermined size. Afterwards, they refill again and the cycle repeats itself.

Landmark Windows^â

[Landmark windows](#) are similar to time-based tumbling windows except you may control the exact time the window expires.

Non-Windowed Metrics^â

Non-windowed metrics are a type of [metric](#) that is not bound by a time window. They are limited only by the available historical data at the time of the metric's creation or update, which is determined by the data retention policy. For more information on this type of metrics, refer to the [Non-windowed Metrics](#) section.

Workflow^â

A workflow defines how a transaction event is routed between processing steps in a fraud detection pipeline.

Each processing step of a workflow is performed by a service (e.g. rules service, scoring service). Thus, a workflow is defined by a list of services that process the transactions and the connections between them.

Please refer to [this section](#) to learn what are workflows and how they work.